

## 1. Explique a diferença entre callbacks, promises e async/await.

### Resposta:

Callbacks são funções passadas como argumento para outra função e executadas quando uma operação assíncrona é concluída. Embora sejam simples, podem gerar o chamado *callback hell*, dificultando a leitura e manutenção do código.

Promises representam o resultado futuro de uma operação assíncrona. Elas permitem tratar sucesso com `.then()` e erros com `.catch()`, tornando o código mais organizado.

Já o **async/await** é uma forma mais moderna de trabalhar com Promises. Ele utiliza uma sintaxe semelhante ao código síncrono, facilitando a leitura, a manutenção e o tratamento de erros com `try...catch`.

## 2. O que é o Worker Threads no Node.js e qual sua utilidade?

### Resposta:

Worker Threads é um módulo do Node.js que permite executar código JavaScript em threads separadas da principal. Sua principal utilidade é realizar tarefas pesadas, como processamento de imagens, cálculos complexos ou criptografia, sem bloquear o Event Loop. Dessa forma, a aplicação continua respondendo normalmente às demais requisições.

## 3. Como o Socket.io facilita a comunicação em tempo real?

### Resposta:

O Socket.io é uma biblioteca que permite comunicação bidirecional em tempo real entre cliente e servidor. Sempre que uma informação é enviada por um dos lados, ela pode ser recebida imediatamente pelo outro, sem necessidade de atualizar a página. É muito utilizado em chats, notificações instantâneas, jogos online e monitoramento em tempo real.

## 4. Qual é o papel dos clusters em aplicações Node.js?

### Resposta:

Clusters permitem que uma aplicação Node.js utilize todos os núcleos do processador, criando múltiplos processos que compartilham a mesma porta de comunicação. Isso melhora o desempenho, aumenta a capacidade de atender múltiplas requisições simultaneamente e torna a aplicação mais escalável.

## 5. Liste três vantagens do PM2 para o gerenciamento de processos.

Resposta:

1. Reinicia automaticamente aplicações que apresentam falhas.
2. Permite monitorar consumo de CPU, memória e desempenho da aplicação.
3. Facilita a execução de aplicações em modo cluster, aproveitando todos os núcleos do processador.

## 6. O que é um child process e como ele é utilizado?

Resposta:

Um child process é um processo filho criado a partir da aplicação principal. Ele é utilizado para executar comandos do sistema operacional ou tarefas independentes, evitando que essas operações bloqueiem o funcionamento da aplicação principal. O Node.js oferece módulos como `child_process` para criar e controlar esses processos.

## 7. Explique o conceito de logging estruturado e sua importância.

Resposta:

Logging estruturado é a prática de registrar informações da aplicação em um formato organizado e padronizado, contendo dados como data, horário, nível do log, mensagem e contexto da operação. Isso facilita o monitoramento, a identificação de erros, auditorias e a análise do comportamento da aplicação em produção.

## 8. Qual é a vantagem de usar o Winston para logs?

Resposta:

O Winston é uma biblioteca que facilita a criação e o gerenciamento de logs. Com ela é possível registrar mensagens em arquivos, no console ou em outros destinos, definir níveis de severidade como *info*, *warn* e *error*, além de organizar melhor as informações para facilitar a manutenção e o diagnóstico da aplicação.

## 9. Por que é importante escalar aplicações Node.js em ambientes de produção?

**Resposta:**

Escalar aplicações permite atender um número maior de usuários simultaneamente, melhorar o desempenho, reduzir o tempo de resposta e aumentar a disponibilidade do sistema. Em ambientes de produção, técnicas como clusters, balanceamento de carga e múltiplos processos ajudam a aproveitar melhor os recursos do servidor e garantem maior estabilidade da aplicação.

**10. Como a programação assíncrona melhora a performance de aplicações?****Resposta:**

A programação assíncrona permite que operações demoradas, como acesso a banco de dados, leitura de arquivos ou chamadas para APIs, sejam executadas sem bloquear o restante da aplicação. Enquanto essas tarefas são processadas, o Node.js continua atendendo outras requisições, aumentando a eficiência, a velocidade de resposta e o desempenho geral do sistema.